Our PCAP project: the parallel approach, explained

Adaptive Scheduling for Load-Balanced Parallel N-Body Simulation: a hybrid MPI + OpenMP + CUDA approach

Tanish Hemant Kuvar 240962540 · M. Vishal Kumar 240962546 · Samyak Jain 240962638 · AIML-A · Manipal Institute of Technology

1 The project in one paragraph

The program repeats one heavy job thousands of times: for every body in a simulation, compute the combined pull of all the other bodies, then move every body a little. (The bodies happen to be stars; that is all the physics you need.) That job is easy to split between workers, which makes it a good parallel computing problem. We split it at **three levels**: across separate programs (**MPI**), across CPU cores (**OpenMP**), and onto the graphics card (**CUDA**). Our own idea is a small **controller** that watches how evenly the work is shared between the CPU threads and changes OpenMP's work-sharing strategy while the program runs.

~6×
faster with 8 threads than 1 (5.6–6.4×)

9/9
uneven cases where the controller beat plain static sharing

≤ 2.3 %
cost of the controller when work is even

88×
graphics card vs 8 CPU threads (exact method)

2 The parallel problem we attack: load imbalance

Work per body is **not equal**. A body sitting in a crowded region needs many more calculations than a body in an empty region. When bodies bunch together, which they do during a run, the work becomes very uneven.

> **Analogy:** 8 people share 65,536 tasks. If each gets the same *number* of tasks and one person happens to get all the long tasks, the other 7 finish early and sit idle. The whole job is only done when the slowest person is done. That idle waiting is **load imbalance**, and it wastes the parallel hardware.

We measure it as **λ (lambda)** = slowest thread's time ÷ average thread time − 1. λ = 0 means everyone finished together; λ = 0.5 means the slowest took 50 % longer than average. In our runs, once bodies bunched up, the busiest thread took **3.7×** as long as the least busy one (λ = 0.63).

3 Our parallel approach, level by level: what, why, and what it gave us

`update own bodies` → `MPI_Allgatherv` → `sort bodies` → `build tree` → `force loop (OpenMP / CUDA)` → `update` → `controller`

One timestep. Every rank (MPI program) runs this; the force loop is where almost all the time goes, so that is what we parallelise and schedule.

LEVEL	WHAT WE PARALLELISED	WHY THIS TECHNIQUE	WHAT IT GAINED US (MEASURED)
OpenMP threads	The force loop: each of 8 threads computes forces for its share of bodies (`#pragma omp for`).	Shared memory on one machine: threads read the same data with no copying, so it is the cheapest way to use all cores.	**5.6–6.4×** faster at 8 threads (N = 65,536), 70–80 % efficiency.
Loop scheduling	How loop iterations are handed out: **static** (equal blocks), **dynamic** (grab chunks of 8 when free), **guided** (big chunks first, then smaller).	Static has zero overhead but cannot react to uneven work; dynamic balances but costs overhead and cache locality. No single choice is best for a whole run.	On uneven inputs every balancing schedule beat static, saving up to **18 %**. On small even jobs dynamic was **46 % slower**.

Adaptive controller (ours)	Chooses the schedule at run time: starts static, switches to dynamic when λ > 0.15, back when λ < 0.05, checking every 5 steps.	Gets static's low cost when work is even and dynamic's balance when it is not, without anyone choosing by hand. Uses only standard OpenMP (`omp_set_schedule` + `schedule(runtime)`).	Faster than static in **9/9** uneven cases (1.5–16.6 %); within **2.3 %** of static on even work.
MPI ranks	Bodies split into blocks between separate programs; after each step they swap positions with `MPI_Allgatherv`.	The only way to go beyond one machine (cluster). Allgatherv because block sizes can differ.	Same answer bit for bit on 1–4 ranks; up to **7.0×** with 16 hardware threads. On one laptop, threads beat ranks.
CUDA GPU	(a) Exact force for all pairs: one GPU thread per body, 256 bodies at a time loaded into fast shared memory ("tiling"). (b) Hybrid: CPU walks the tree, GPU does the close-range pairs.	The force maths is the same simple operation repeated billions of times, which is ideal for thousands of GPU cores. Tiling reuses data from fast on-chip memory.	Exact method **88×** faster than 8 CPU threads (24 vs 2,117 ms per step). Hybrid **1.3–1.4×** faster than CPU only.

Design decisions that make the parallel parts work

- **Fewer calculations first (Barnes–Hut tree).** Far-away groups of bodies are treated as one, cutting the work from N^2 to about N log N. Parallelism then speeds up a smaller job.
- **Morton sort every step.** Bodies are sorted along a zig-zag space-filling curve so neighbours sit next to each other in memory: better cache use and a fast tree build. Side effect: each thread's block is one region of space, which is exactly why bunching causes imbalance.
- **No race conditions.** In the force loop each thread only writes its own bodies' results; there are no locks or shared writes, so threads never wait on each other inside the loop.
- **Identical results everywhere.** Each body's force is computed in the same order whoever computes it, so 1 thread, 8 threads, 4 ranks, static or adaptive all give a bit-for-bit identical answer (checked across 13 configurations).
- **Thread pinning.** The laptop CPU has 8 fast and 12 slow cores. Unpinned threads landing on slow cores looked 30 % imbalanced on perfectly even work, so we pin one thread per fast core.
- **Precise timing.** The compiler's OpenMP timer only measured in 1 ms steps, too coarse for per-thread times, so we use the Windows high-resolution clock.
- **CUDA without Visual Studio.** The GPU code is compiled while the program runs (NVRTC), so it needs no CUDA compiler install and still runs on machines without a GPU.

4 The controller, and the fix we had to make

Every 5 steps the controller adds up each thread's loop time, computes λ, and applies the rule above with `omp_set_schedule(policy, 8)`. Using two thresholds (0.15 to switch, 0.05 to switch back) is called **hysteresis**: it stops the controller flipping back and forth on small timing noise.

> **The flaw we found and fixed.** Under dynamic, threads keep grabbing work until it runs out, so they always finish together and the measured λ is always about 0. The rule "λ < 0.05 → switch back" would then fire immediately, static would be unbalanced again, and the controller would oscillate forever. **Our fix:** while dynamic is on, the controller *predicts* how long each thread would take under static, from the work in its static block and the thread's measured speed (predicted time = block work ÷ speed). Under static this equals the measured time, so nothing changes there.

5 What we found (honestly)

- **The synopsis prediction held:** the controller matches static when work is even and beats it when work is uneven (all 9 cases, by up to 16.6 %).

- **But OpenMP's built-in *guided* schedule was as good or better** (best in 9 of 12 cases). Guided's big first chunks keep neighbouring bodies together (good for the cache) and its small last chunks fix the imbalance; our controller wastes the early part of each run in static. So the controller's value is that **you never have to pick a schedule by hand**.
- **Timing noise can fool it:** once, a random spike made it switch on even work, 17 % slower than static.
- **MPI on one laptop:** every extra rank repeats the sort and tree build and must exchange data, so 1 rank × 8 threads was the best use of 8 cores. Imbalance between ranks (up to λ = 0.11) is something the OpenMP controller cannot fix. Vishal's `--partition cost` splits ranks by measured work instead: up to 17.2 % faster with 4 ranks.
- **GPU precision:** this laptop GPU is about 64× slower in double precision, so single precision (error 0.0005 %) is what makes it fast.
- **Measuring fairly:** 3 repetitions in shuffled order, median reported, and a "thermal gate" that waited for the laptop to cool, because it slowed by up to 40 % when hot.

---

6 Running the demo

Anaconda Prompt → `cd "E:\BTech 3rd Year (Sem 5)\PCAP-Project"` → `python demo.py` (Enter between parts; `--part N` runs one part). Part 1: same answer for 1 and 8 threads. Part 2: equal shares vs our controller, with bars showing λ and the moment it switches. Part 3: 1 × 8 vs 4 × 2 MPI, same answer. Part 4: GPU. Part 5: opens the results page with the animation (works offline). New machine: `python build.py --fetch-cuda`, then `python scripts/validate.py`.

---

7 Viva questions and answers (parallel computing)

What did you parallelise, and why that part?
The force loop: it is almost all the run time and each body's force can be computed independently.

What is load imbalance? How do you measure it?
Some threads get more work, so the rest wait. λ = max thread time ÷ mean − 1; 0 is perfect.

Static vs dynamic vs guided?
Static: equal blocks fixed up front, no overhead. Dynamic: chunks handed out on demand, balanced but with overhead. Guided: chunks start big and shrink, low overhead plus balance at the end.

How do you change the schedule at run time?
The loop uses `schedule(runtime)`; `omp_set_schedule()` sets which schedule it uses next time. No recompiling.

Why not just always use dynamic?
Overhead and lost cache locality: on a small even job it was 46 % slower than static. Chunk size 1 was even slower than static on uneven work.

Why two thresholds?
Hysteresis, so noise near one threshold doesn't make it switch back and forth.

What was wrong with the original algorithm?
Under dynamic, measured times are always balanced, so it would switch back immediately and oscillate. We predict static's times instead.

Why didn't your controller beat guided?
It reacts only after imbalance appears, and guided has tiny overhead while keeping locality. It still beat static every time work was uneven.

Why only ~6× with 8 threads, not 8×?
Sorting and tree building ran on one thread (about 8 % of a step; Amdahl's law), memory bandwidth limits, and laptop noise. Vishal later parallelised the sort and tree (~2× faster).

Any race conditions?
No: each thread writes only its own bodies' results. No shared writes in the force loop.

Why pin threads?
Fast and slow cores on the CPU; a thread on a slow core looks "behind" even with equal work.

How does MPI split the work?
Each rank owns a block of bodies; after each step `MPI_Allgatherv` shares all positions so every rank can build the full tree and compute its own block.

Why Allgatherv, not Allgather?
Blocks can have different sizes (N not divisible by the rank count, or cost-based splitting).

Why is 1 rank × 8 threads faster than 4 × 2?
Threads share memory for free; ranks repeat the sort and tree, exchange data every step, and can be imbalanced with each other.

How does the CUDA kernel work?
One GPU thread per body; a block of 256 threads loads 256 bodies into shared memory, all use them, then the next tile (tiling).

Why single precision on the GPU?
The laptop GPU is ~64× slower in double. Single precision error (0.0005 %) is far below the method's own 0.25 %.

How do you know parallel results are correct?
8 automatic checks; the final state is bit-identical across 13 thread/rank/schedule configurations, and the GPU matches the CPU.

How did you keep measurements fair?
Pinned threads, 3 shuffled repetitions with medians, and a thermal gate; throttled data discarded.

What would you do next?
Balance work between ranks too (done: `--partition cost`), let the controller also try guided (done: `--balanced auto`), run on a real multi-node cluster.

Who did what?
[Agree as a team before the viva.]